DevOps-Based Deployment and Orchestration

of a Multi-Agent Insurance System

Course Name: DevOps

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mustafa shaikh	EN22CS301624
02	Naman Mehta	EN22CS301629
03	Naman Raghuvanshi	EN22CS301632
04	Nandini Sharma	EN22CS301635
05	Nishkarsh Sharma	EN22CS301659

Group Name: DO-20

Project Number: 09D6

Industry Mentor Name: Vaibhav Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

Orchestrate the deployment of a Insurance application onto a Kubernetes cluster (Minikube). The implementation includes defining robust deployment and service manifests to manage containerized workloads. A key feature is the implementation of zero-downtime 'Rolling Updates', ensuring seamless transitions between image versions while maintaining service availability for end-users.

2. Project Objectives

The objective of the Multi-Agent Insurance Recommendation System is to develop a web-based application that helps users identify suitable insurance policies based on their personal and financial requirements. The system utilizes a multi-agent approach to validate inputs, analyze policy data, generate recommendations, and provide comparisons for better decision-making.

- Build a web-based Multi-Agent Insurance System for policy recommendation.
- Simulate multiple agents (Input, Recommendation, Comparison, Query) to modularize decision flow.
- Provide personalized top policy suggestions based on user profile and preferences.
- Display comparison of shortlisted plans in a clear tabular format.
- Add a basic chatbot for rule-based insurance query handling.
- Containerize the application using Docker for consistent execution.
- Deploy the app on Kubernetes (Minikube) to demonstrate orchestration fundamentals.

The overall goal is to create a user-friendly, scalable, and easily deployable solution with modern DevOps and cloud integration.

3. Scope of the Project

The scope of the Multi-Agent Insurance Recommendation System includes the design and development of a web-based application that provides personalized insurance recommendations using a multi-agent approach.

The project covers:
• User interaction through a web interface for submitting insurance-related queries
• Processing of user input using multiple agents for validation, recommendation, comparison, and query handling
• Retrieval of insurance policy data from a JSON-based data source
• Display of recommended policies and comparison results in a structured format

- From a DevOps perspective, the scope also includes:
- **In scope**
  - Flask-based backend APIs (/recommend, /compare, /chat)
  - • Version control and code management using Git-based repositories
  - Frontend form + result views + simple chat interface
  - JSON-based policy data source
  - Docker image/container creation
  - Kubernetes Deployment and Service for local cluster-based hosting

The project is limited to a simulated environment with predefined policy data and rule-based responses, and does not include real-time integration with external insurance providers.

Proposed Solution

Key features

- User profile input validation
- Policy recommendation engine (top matches)
- Policy comparison table
- Rule-based insurance assistant/chat
- Responsive web interface
- Dockerized deployment
- Kubernetes-based local orchestration (Deployment + Service)

Overall Architecture / Workflow

The Multi-Agent Insurance Recommendation System follows a layered architecture consisting of frontend, backend, multi-agent processing, data storage, and DevOps deployment components.

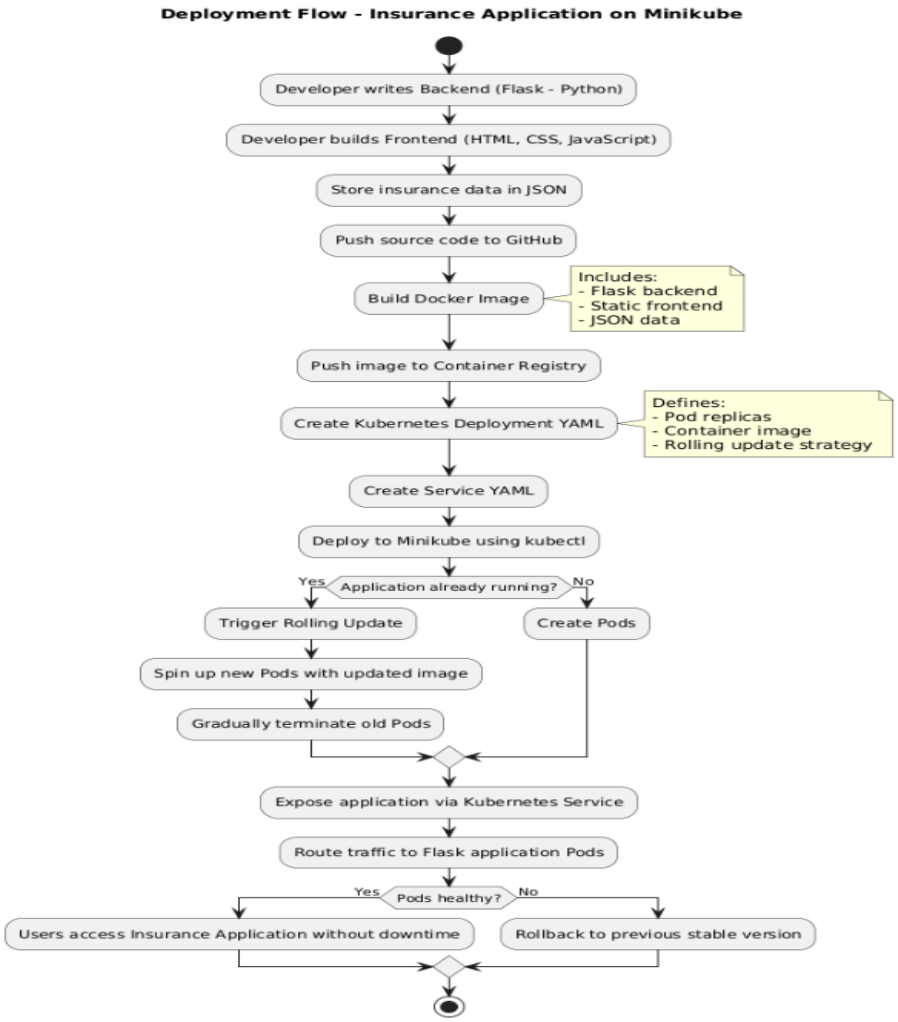
The **frontend layer** provides a web-based interface where users enter their details and view recommended insurance policies. The **backend layer**, implemented using Flask, handles user requests and coordinates the processing logic.

The core functionality is managed by the **multi-agent system**, where different agents perform specific tasks. The Input Agent validates user data, the Recommendation Agent selects suitable policies, the Comparison Agent generates a comparison table, and the Query Agent handles user queries.

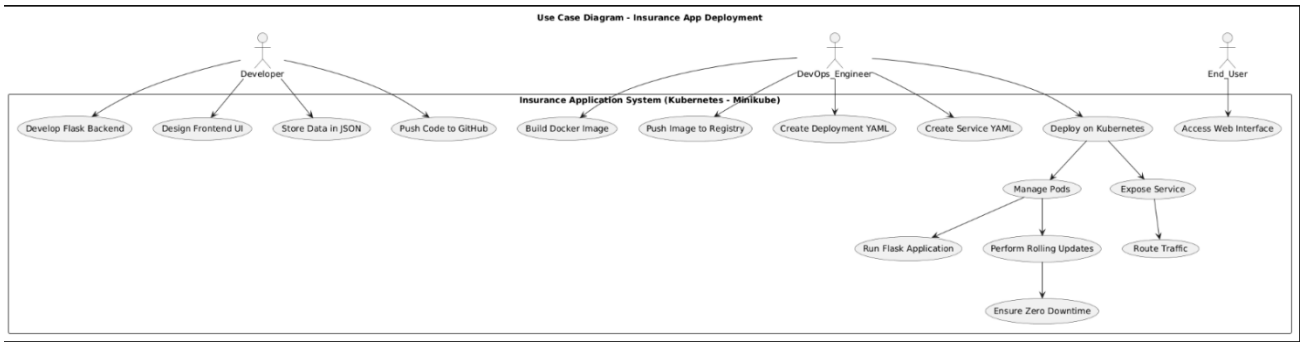
The system retrieves policy data from a JSON-based data store and processes it to generate structured outputs for the user.

From a DevOps perspective,

1. User enters details (age, income, family members, risk preference, coverage).
2. **Input Agent** validates and normalizes data.
3. **Recommendation Agent** scores policies from policies.json and returns top candidates.
4. **Comparison Agent** structures shortlisted policies for side-by-side comparison.
5. **Query Agent** handles basic user questions using predefined rule-based responses.
6. Backend returns data to frontend for cards/table/chat rendering.
7. App is packaged via Docker and deployed as pods via Kubernetes Service exposure.



Use case Diagram:



Tools & Technologies Used

• **Backend** – Flask (Python-based framework for handling application logic and API requests)

• **Frontend** – HTML, CSS, JavaScript (for building the user interface)

• **Programming Language** – Python

• **Data Storage** – JSON (used to store insurance policy data)

• **Version Control** – GitHub (for source code management)

• **Containerization** – Docker (for packaging the application)

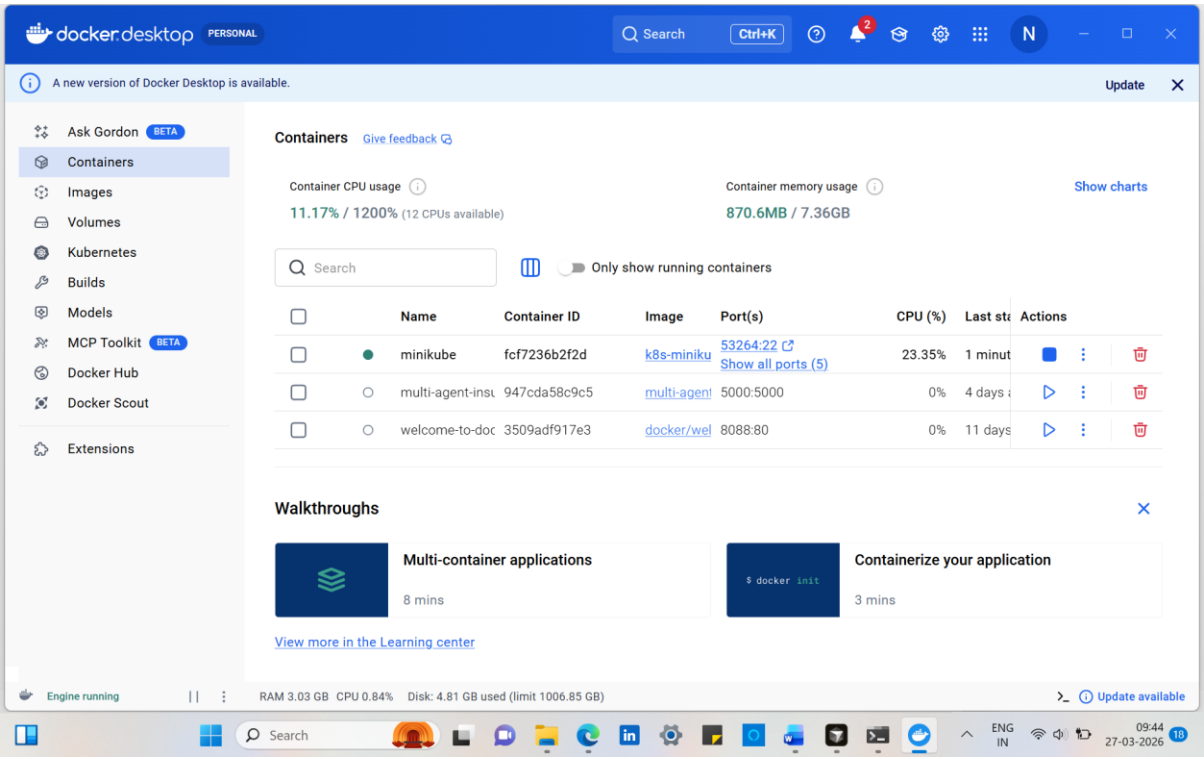
• **Infrastructure as Code (IaC)** – Terraform

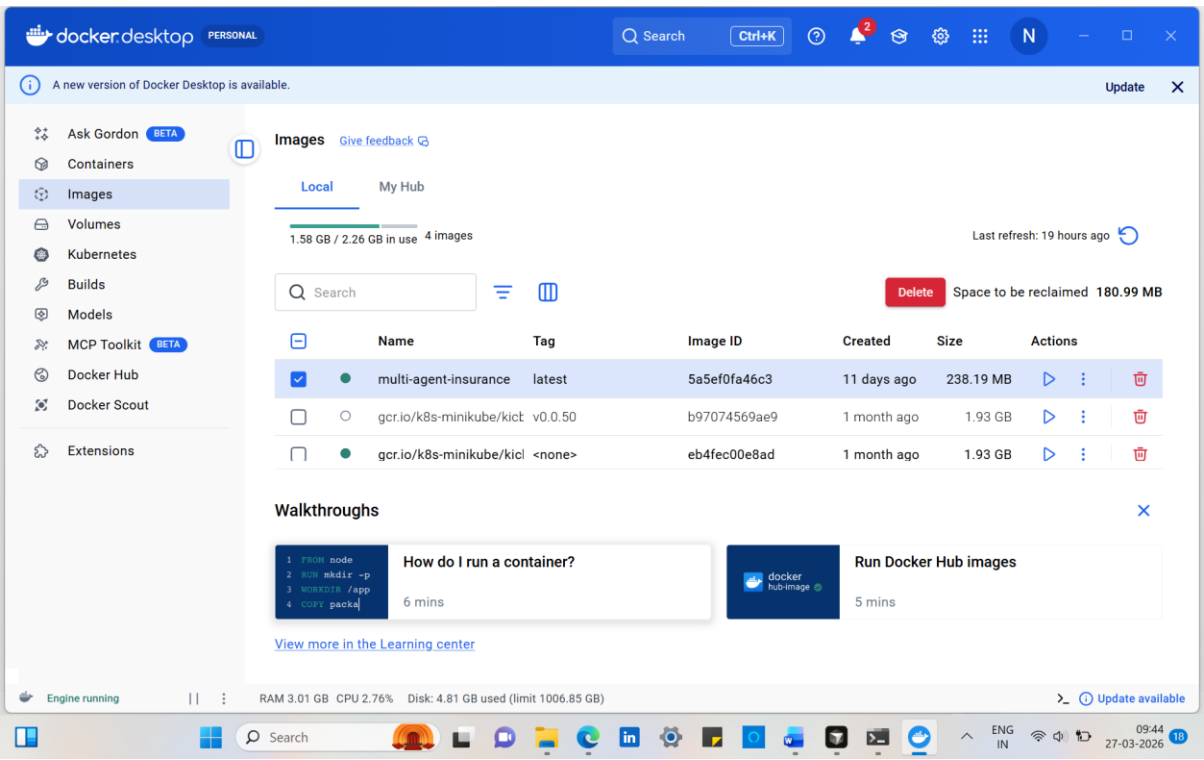
• **Cloud Platform** – AWS Management Console (for deploying and managing infrastructure)

• **Container Orchestration** – Kubernetes (for managing containerized applications)

Results & Output

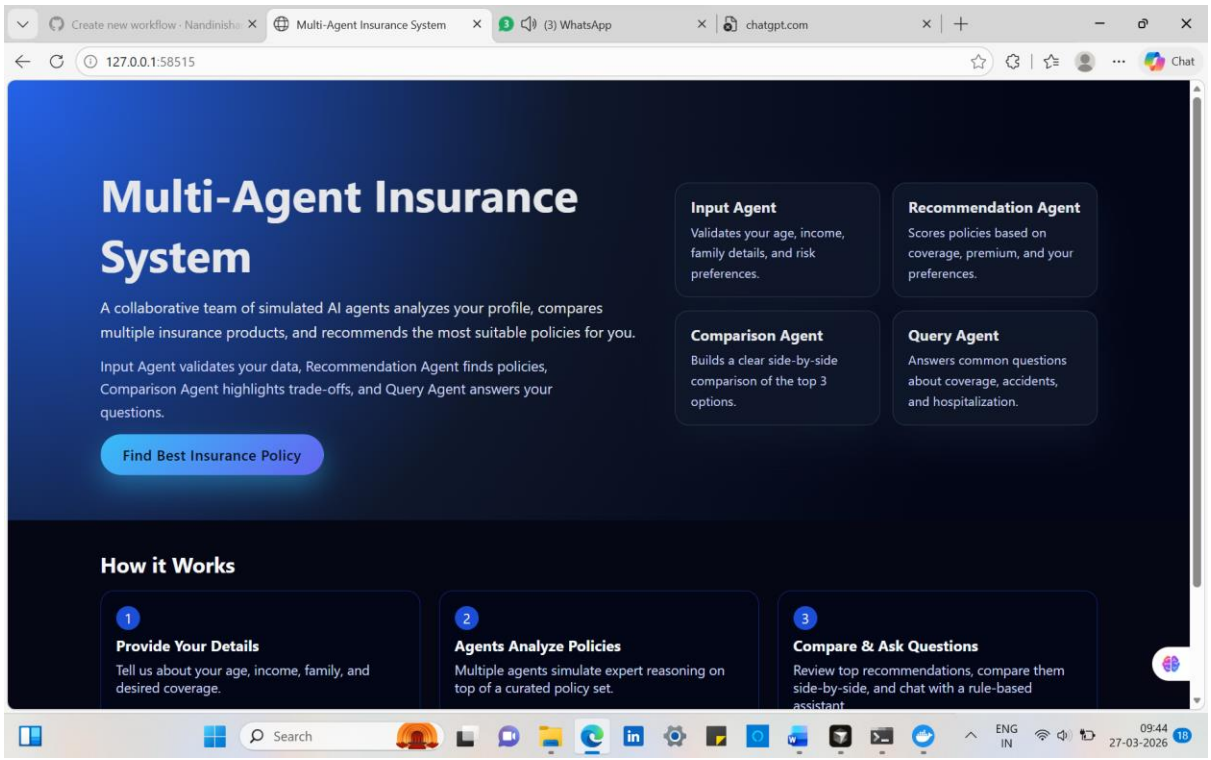
Docker Container and image:



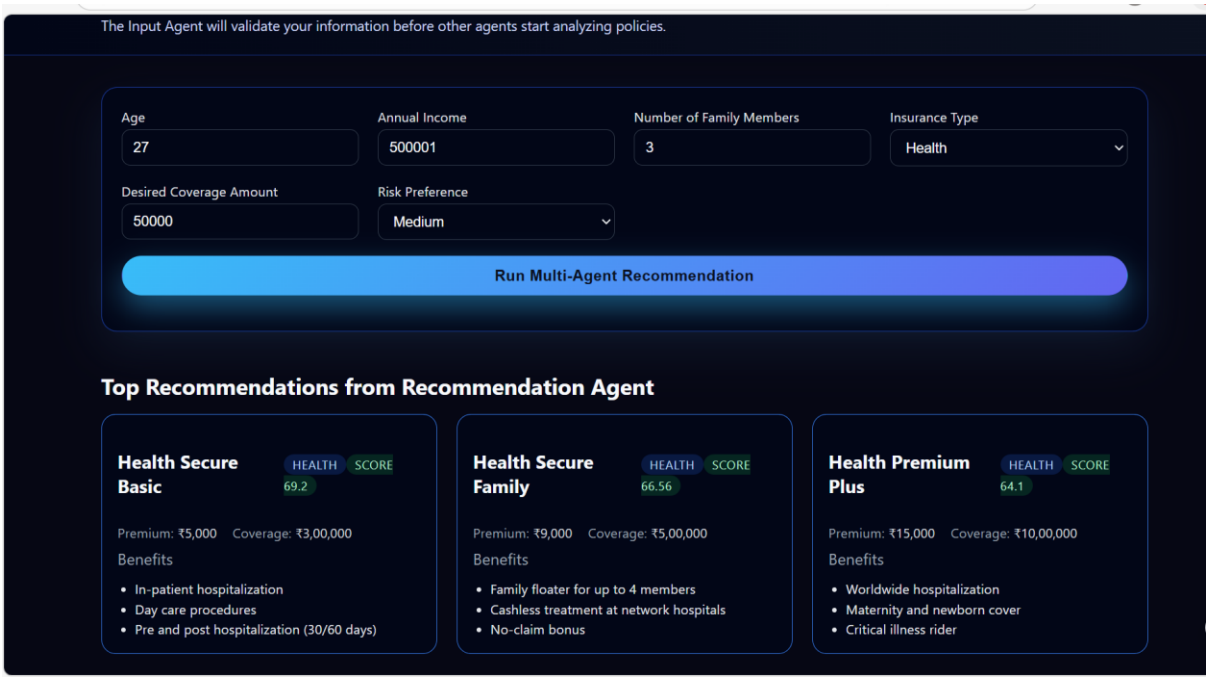


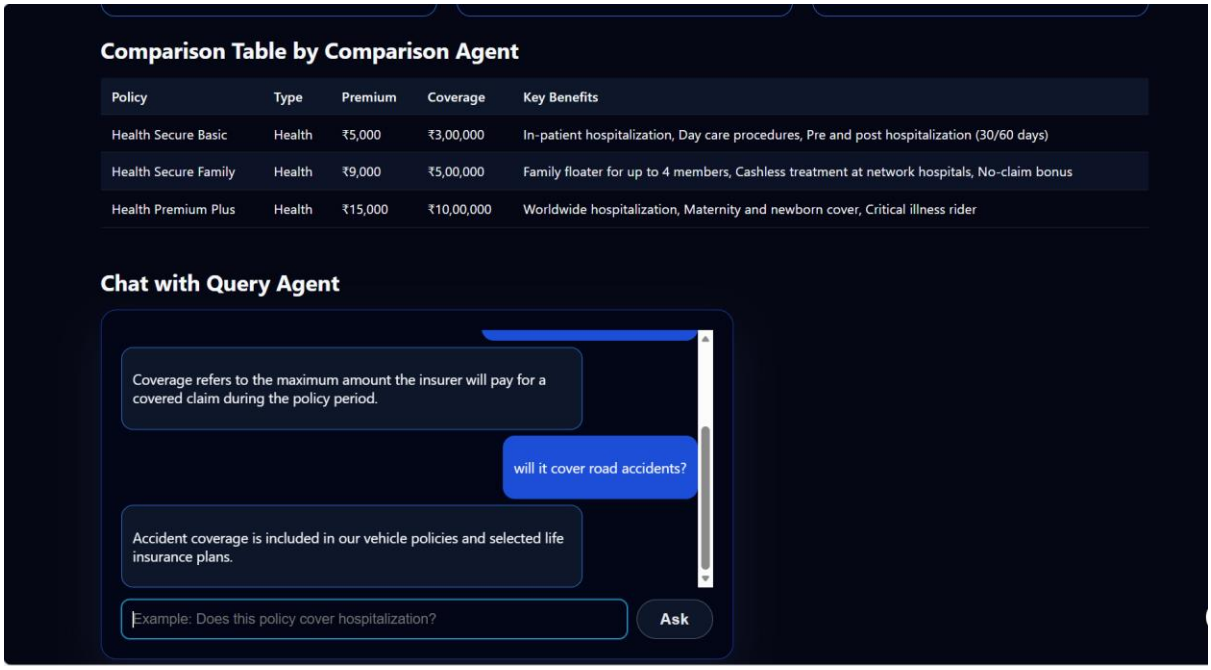
Kubernetes:

```
PS C:\Users\lenovo> minikube start --driver=docker
😄  minikube v1.38.1 on Microsoft Windows 11 Home Single Language 21H2
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
🔄  Restarting existing docker container for "minikube" ...
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\lenovo> minikube service insurance-app-service --url
http://127.0.0.1:53313
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```



Working:


The Input Agent will validate your information before other agents start analyzing policies.
Age
27
Annual Income
500001
Number of Family Members
3
Insurance Type
Health
Desired Coverage Amount
50000
Risk Preference
Medium
Run Multi-Agent Recommendation
Top Recommendations from Recommendation Agent
Health Secure Basic
HEALTH
SCORE
69.2
Premium: ₹5,000 Coverage: ₹3,00,000
Benefits
In-patient hospitalization
Day care procedures
Pre and post hospitalization (30/60 days)
Health Secure Family
HEALTH
SCORE
66.56
Premium: ₹9,000 Coverage: ₹5,00,000
Benefits
Family floater for up to 4 members
Cashless treatment at network hospitals
No-claim bonus
Health Premium Plus
HEALTH
SCORE
64.1
Premium: ₹15,000 Coverage: ₹10,00,000
Benefits
Worldwide hospitalization
Maternity and newborn cover
Critical illness rider


Comparison Table by Comparison Agent
Policy | Type | Premium | Coverage | Key Benefits
Health Secure Basic | Health | ₹5,000 | ₹3,00,000 | In-patient hospitalization, Day care procedures, Pre and post hospitalization (30/60 days)
Health Secure Family | Health | ₹9,000 | ₹5,00,000 | Family floater for up to 4 members, Cashless treatment at network hospitals, No-claim bonus
Health Premium Plus | Health | ₹15,000 | ₹10,00,000 | Worldwide hospitalization, Maternity and newborn cover, Critical illness rider
Chat with Query Agent
Coverage refers to the maximum amount the insurer will pay for a covered claim during the policy period.
will it cover road accidents?
Accident coverage is included in our vehicle policies and selected life insurance plans.
Example: Does this policy cover hospitalization?
Ask

Key Outcomes

• Successful development of a multi-agent system for insurance recommendation

• Accurate and personalized policy suggestions based on user input

• Efficient comparison of insurance policies for better decision-making

• Implementation of a user-friendly web interface

• Integration of DevOps practices for automated build and deployment

• Containerization of the application using Docker

• Version control and code management using GitHub

• Infrastructure provisioning using Terraform

• deployment using Kubernetes

• Improved scalability and maintainability through automation and container orchestration

Conclusion

This project demonstrates an end-to-end implementation of a Multi-Agent Insurance System using a Flask backend, web frontend, and JSON policy data. The modular agent flow improved clarity of responsibilities across validation, recommendation, comparison, and query handling. Beyond application logic, the project emphasized DevOps practices: Docker for packaging, Kubernetes for orchestration, and Terraform for infrastructure learning on AWS. Overall, key learning includes designing structured backend workflows, deploying containerized applications reliably, and troubleshooting real-world environment issues across local and cloud-oriented tooling.

Future Scope & Enhancements

• Integration with real-time insurance provider APIs for live policy data

• Implementation of advanced AI/ML models to improve recommendation accuracy

• Enhancement of the chat assistant using NLP for more intelligent responses

• Addition of user authentication and personalized user profiles

• Migration from JSON to a scalable database system (e.g., SQL/NoSQL)

• Deployment using advanced cloud services and auto-scaling features on the AWS Management Console

• Implementation of monitoring and logging tools for better system observability

• Enhancement of CI/CD pipelines using tools like Jenkins for advanced automation

• Support for mobile-friendly or cross-platform applications